

Harness

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon 1

Preface 2

Who this book is for 2

A book that stands on its own 2

What you need 3

Conventions 3

I: The Why 4

1. Tests that leave a trace 5

1.1. The thread exists 5

1.2. In one JVM it is free 5

1.3. What a trace is for 6

1.4. Where this comes from 6

2. A small module 7

2.1. Prior art 7

2.2. Decisions 7

II: The How 9

3. Getting started 10

3.1. Adding the module 10

3.2. The module 10

3.3. Tests on the module path 10

4. A server for the length of a body 12

4.1. In a server 12

4.2. The server 13

4.3. Up while the body runs 13

4.4. Serving files 14

5. One trace 15

5.1. Recording a test 15

5.2. The tree 15

5.3. Failures 16

5.4. Other clients 17

6. Recording 18

6.1. What it is 18

6.2. Building one 18

6.3. Using one 18

III: Looking Ahead **20**
7. What comes next **21**
7.1. Browsers 21
7.2. Documents from traces 21
7.3. Beyond one JVM 21
Appendix A: Reference **22**
A.1. Types 22
A.2. Spans 22
Appendix B: Building this book **23**
Appendix C: Glossary **24**

Colophon

Harness: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the harness module of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime:harness:2026.11.0`, Java module `org.nasdanika.sdk.runtime.harness`, and the `Recording` class of the runtime's telemetry model exporters. Every listing in it is included from the module's sources and tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Harness: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Java is a trademark of Oracle and its affiliates. OpenTelemetry is a trademark of the Linux Foundation. Eclipse and EMF are trademarks of the Eclipse Foundation. Playwright is a trademark of Microsoft Corporation. Spring is a trademark of Broadcom. Other names may be trademarks of their owners. Their use here identifies the technologies discussed and implies no endorsement.

The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/runtime, in `harness`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about running code against an HTTP server and keeping a record of what both sides did. It describes `harness`, a small module of the Nasdanika SDK Runtime with one method at its heart: `Harness.inServer`, which starts a server, runs your code while the server is up, and stops it. Everything that happens in between, the requests your code sends and the work the server does to answer them, is one OpenTelemetry trace, and with the runtime's telemetry model the trace is saved as a model you can query, compare and generate documents from.

The first part is the why: why tests should leave a trace, and why the module is as small as it is. The second part is the how. The third says what is built on it and what comes next.

Who this book is for

Java developers who test web applications and services, and want a failed test to say what the server did, not only what the client got.

Developers who build demos and walk-throughs of web applications, and want them reproducible and documented by the same run.

Architects who want end-to-end traceability from a requirement, through a test, to the code that served it. Part one is for you, and needs no code.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. The runtime has a book of its own, *Nasdanika SDK Runtime: More Structure with Less Effort*, and the browser side of the same idea has a Zephyr Book, *Walk-throughs with Playwright*. You do not need either to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The module is `org.nasdanika.sdk.runtime.harness`, Maven coordinates `org.nasdanika.sdk.runtime:harness:2026.11.0`.
- To record telemetry as models, the runtime's telemetry model exporters, `org.nasdanika.sdk.runtime.models.telemetry:exporters`, which bring the OpenTelemetry SDK.

Conventions

- **Every listing is included from the module's sources or tests** and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: **PLANNED** for designed but not built.
- **Voice.** I write in the first person.



> Editor's notes record inconsistencies and claims to verify found while writing. They are shown in builds for the author and left out of published editions.

Part I: The Why

Why tests should leave a trace, and why this one is small. No code is needed to read this part.

Chapter 1. Tests that leave a trace

An integration test of a web application sends requests and checks responses. When it passes, it leaves nothing behind but a green mark. When it fails, it leaves an assertion message on one side and, somewhere else, the server's logs on the other, and someone has to put the two together: which request was this, what did the server do with it, where did it go wrong.

The information was there while the test ran. The client knew which request it was sending and why; the server knew what it did to answer. What was missing is the thread that ties them together.

1.1. The thread exists

Distributed tracing is that thread. OpenTelemetry, the standard for it, gives every unit of work a trace and every operation in it a span, and carries the trace from process to process in a request header, `traceparent`, in the W3C format. A server that reads the header makes its span a child of the client's span, and the two sides become one tree.

Production systems use it to follow a request through dozens of services. Tests rarely do, and the reason is mostly setup: a collector, a backend to store traces in, an agent or an SDK configured on both sides, and a way to find the test's trace among everything else.

1.2. In one JVM it is free

A test that starts the server it tests, in the same JVM, needs none of that. There is one OpenTelemetry instance, so there is one SDK, one exporter and one place the telemetry goes. If the client sends `traceparent` and the server reads it, every request of the test and everything the server did for it are in the test's trace, automatically, and nothing else is.

The harness makes that the default. `Harness.inServer` starts a server whose every handler is instrumented, runs your code in a span, and gives it a client whose requests are spans that carry the trace. The runtime's telemetry model saves the trace as a model: spans with their children, attributes and log records, in YAML, JSON or XMI.

1.3. What a trace is for

A trace recorded by a test is evidence, and evidence has several readers.

The developer of a failing test sees the request, the server span it caused, the log records the server emitted and the exception it recorded, in one tree.

The reviewer sees what a change did to the work behind a test: a new request, a request that got slower, a resource loaded twice. Traces are models, so two runs can be compared like any two models.

The writer of documentation gets a description of a scenario that is true by construction: the steps a test took, what each step asked the server for, and what the server did. With screenshots from a browser, which the runtime's Playwright module adds, the trace is a walk-through. Generated into AsciiDoc, a slide deck or a sequence diagram, it is documentation that cannot drift from the system, because the next build regenerates it.

1.4. Where this comes from

In an earlier generation of Nasdanika, WebTest ran browser tests and reported every action with screenshots taken before and after it, so a test report was also a picture book of the application. What it could not show was the server's side. This module and the Playwright module after it are the same idea with the missing half added: the browser's steps, the screenshots and the server's work, in one trace.



Describe Nasdanika WebTest from its repository, github.com/Nasdanika/webtest, before publishing: what it was built on and what its reports contained. The text assumes browser tests with screenshots before and after each action, as the author recalls it.

Chapter 2. A small module

The harness is four classes, two of them public, in a few hundred lines of Java. That is a design decision, and this chapter explains it, starting with what already exists.

2.1. Prior art

Framework test support. Spring Boot starts an application on a random port for a test, `@SpringBootTest` with `RANDOM_PORT`, and gives the test a client for it; with Micrometer Tracing configured, both sides can report to the same tracer. It is excellent inside a Spring application, and it is a Spring application's harness.

Testcontainers runs the real dependencies of a test, databases, brokers and other services, in containers. It answers a different question, "what does my code talk to", and works well together with tracing, at the cost of containers.

Trace-based testing. Tracetest runs a request against a deployed system and asserts on the trace it produced, read back from a tracing backend. OpenTelemetry's own `InMemorySpanExporter` lets a unit test assert on the spans of code it calls. Both treat the trace as something to check; the harness treats it as something to keep as well.

Playwright's trace viewer records what a browser did, with screenshots, DOM snapshots and network calls, in a format of its own for its own viewer. It sees the browser's side only.

2.2. Decisions

The JDK's server, not a framework. `com.sun.net.httpserver.HttpServer` is part of the JDK, handles requests on virtual threads, and is what the runtime serves volumes and generated applications with. A test configures it the way it configures any `HttpServer`, with `createContext`; there is nothing else to learn.

Shaped like `inSpan`. The runtime runs code in a span with `Telemetry.inSpan(tracer, name, attributes, body)`: the span starts, the body runs, a failure is recorded, the span ends. `Harness.inServer(name, routes, body)` has the same shape and the same guarantees, with a server around the body. Generic in what the body returns and what it throws, so the body is ordinary code, assertions included.

Instrumentation that cannot be forgotten. The code that configures the server receives a server whose `createContext` adds the runtime's telemetry filter to every context it creates. A handler cannot be left out of the trace by accident.

One instance, given or current. The harness does not configure OpenTelemetry; it uses the instance it is given, or the current one, the same instance for the client and the server. Without an SDK it costs nothing and propagates nothing; with one, the trace goes wherever the SDK sends it.

Recording is separate. Saving telemetry as models is the job of `Recording`, in the telemetry model's exporters, which the runtime's command line launcher uses for every command it runs. The harness does not depend on it, so a test that only needs a server does not bring the OpenTelemetry SDK with it.

Browsers are separate too. The harness's client is the JDK's `HttpClient`. Browsers belong to the Playwright module, which follows the same pattern one level up: a session runs in the server's trace, and its steps are spans.

Part II: The How

Adding the module, running code against a server, reading the trace it leaves, and recording the trace as a model.

Chapter 3. Getting started

3.1. Adding the module

The runtime has a bill of materials, `org.nasdanika.sdk.runtime:bom`, which manages the versions of its modules and their dependencies. Import it in `dependencyManagement` with the scope `import` and the type `pom`, and depend on `org.nasdanika.sdk.runtime:harness` without a version, usually in the `test` scope.

To record telemetry as models, add the runtime's telemetry model exporters, `org.nasdanika.sdk.runtime.models.telemetry:exporters`, which bring the OpenTelemetry SDK, and, for YAML or JSON files, the runtime's `emf-json` module.

3.2. The module

Listing 1. The module declaration (`module-info.java`)

```
/**
 * An HTTP server that is up while code runs, in one trace with the code's requests.
 */
module org.nasdanika.sdk.runtime.harness {

    requires transitive org.nasdanika.sdk.runtime.volume; // TelemetryHandler, jdk.httpserver,
java.net.http

    exports org.nasdanika.sdk.runtime.harness;

}
```

The harness requires the runtime's `volume` module transitively, for its telemetry handler and its HTTP handler for volumes, and with it the JDK's HTTP server and client and the runtime's `common` module. A module that requires the harness reads all of them.

3.3. Tests on the module path

Tests patched into a module on the module path read what the module reads. Maven's Surefire plugin puts the modules a test needs but the main module does not on the module path when it can, and on the class path otherwise: the telemetry model, for example, goes on the module path, and its exporters, which need the OpenTelemetry SDK, on the class path. A test that reads

a recording then needs one more read edge, added in the Surefire configuration with `--add-reads <your module>=org.nasdanika.sdk.runtime.models.telemetry`. The module's own tests do exactly that.

Chapter 4. A server for the length of a body

4.1. In a server

`Harness.inServer` takes a name, the routes of a server and a body:

Listing 2. A greeting (`ServerTests.java`)

```
String greeting = Harness.inServer(
    "Greeting", ①
    routes -> routes.createContext("/hello", ex -> respond(ex, 200, "Hello!")), ②
    server -> {
        HttpResponse<String> response = server.get("/hello"); ③
        assertThat(response.statusCode()).isEqualTo(200);
        return response.body(); ④
    });
assertThat(greeting).isEqualTo("Hello!");
```

① The name of the span the body runs in.

② The routes: code that creates the server's contexts, given the server as an `HttpServer`.

③ A request to the server, a client span in the body's span.

④ What the body returns, `inServer` returns.

The handler in the routes is ordinary JDK code:

Listing 3. A plain text response

```
static void respond(HttpExchange exchange, int status, String text) throws IOException {
    byte[] bytes = text.getBytes(StandardCharsets.UTF_8);
    exchange.getResponseHeaders().set("Content-Type", "text/plain; charset=utf-8");
    exchange.sendResponseHeaders(status, bytes.length);
    try (OutputStream body = exchange.getResponseBody()) {
        body.write(bytes);
    }
}
```

The server listens on the loopback address, on a port the operating system picks, so tests never collide on a port and nothing outside the machine can reach the server. It handles each request on a virtual thread. Every context created in the routes is instrumented: each request is a server span named after the method and the context path, `GET /hello`, with the HTTP semantic convention attributes, and the span is current while the handler runs.

The routes receive the server as an `HttpServer` for creating contexts. Starting, stopping and the executor belong to the harness, and the server refuses them.

4.2. The server

The body receives a `Server`:

Method	Returns
`base()`	`http://127.0.0.1:<port>/`
`port()`	The port
`uri(path)`	The base resolved against a path, with or without a leading slash
`get(path)`	A `GET` of the path, with the body as a string
`send(request, bodyHandler)`	The response to any request, sent in a client span
`client()`	The `HttpClient` behind `get` and `send`
`span()`	The span the body runs in
`openTelemetry()`	The instance the server and the client report to
`propagationHeaders(context)`	The headers that carry a context, for clients the harness does not own
`httpServer()`	The JDK server itself

4.3. Up while the body runs

The server is stopped when the body returns or throws:

Listing 4. Nobody listens afterwards

```
URI base = Harness.inServer("Stopping", routes -> {}, server -> server.base());
try (HttpClient client = HttpClient.newHttpClient()) {
    HttpRequest request = HttpRequest.newBuilder(base).build();
    assertThatThrownBy(() -> client.send(request, HttpResponse.BodyHandlers.ofString()))
        .isInstanceOf(ConnectException.class);
}
```

4.4. Serving files

Any `HttpHandler` can be routed. The runtime's volume handler serves a directory, an in-memory tree, or anything else a volume is implemented over:

Listing 5. A directory, served

```
String readme = Harness.inServer(
    "Files",
    routes -> routes.createContext("/files", new VolumeHttpHandler(Volume.of(dir))),
    server -> server.get("/files/readme.txt").body());
assertThat(readme).isEqualTo("Read me");
```

The request path below the context path is the path in the volume. Static sites, generated applications and their data are served this way in the runtime's own tests.

Chapter 5. One trace

The examples so far ran with the current OpenTelemetry instance, which in a plain test is the no-op one: spans are not recorded and no trace context is sent. With an SDK, the same code leaves a trace.

5.1. Recording a test

A `Recording` is an OpenTelemetry SDK that exports to the runtime's telemetry model and saves the model when it is closed (Chapter 6):

Listing 6. A recorded test (`TraceTests.java`)

```
Recording recording = Recording.builder()
    .serviceName("harness-tests")
    .directory(dir, "yml") ①
    .build();
try (recording) {
    Harness.inServer(
        recording.getOpenTelemetry(), ②
        "Library",
        routes -> routes.createContext("/books", exchange -> {
            var logger = Telemetry.logger(Telemetry.current()); ③
            Telemetry.log(logger, Severity.INFO, null, "Listing books", null);
            respond(exchange, 200, "Dune, Solaris");
        }),
        server -> server.get("/books"));
} ④
```

① The model is saved as `traces.yml`, `logs.yml` and `metrics.yml` in the directory.

② The instance the server and the client report to.

③ In a handler, the current instance is the server's, and the request's span is current, so the log record goes into the request's span.

④ Closing the recording exports what is pending and saves the files.

5.2. The tree

The recording holds one trace, a tree:

Listing 7. The trace of the test

```
Span library = roots(recording).getFirst();
assertThat(library.getName()).isEqualTo("Library");

Span client = library.getChildren().getFirst(); ①
assertThat(client.getName()).isEqualTo("GET /books");
assertThat(client.getKind()).isEqualTo(SpanKind.SPAN_KIND_CLIENT);

Span server = client.getChildren().getFirst(); ②
assertThat(server.getName()).isEqualTo("GET /books");
assertThat(server.getKind()).isEqualTo(SpanKind.SPAN_KIND_SERVER);
assertThat(server.getTraceId()).isEqualTo(library.getTraceId());

assertThat(server.getLogRecords()) ③
    .singleElement()
    .satisfies(log -> assertThat(log.getBody().getStringValue()).isEqualTo("Listing books"));
```

① The body's span, `Library`, has the client span of the request as a child.

② The client span has the server's span as a child: the server read the trace context from the request headers.

③ The handler's log record is in the server span.

The same tree is in `target/telemetry/one-trace/traces.yml`, with timestamps, attributes and identifiers: the body span with the server's address and port, the client span with the method, the URL and the response status, and the server span with the route, the path, the client's address, the user agent and the response size.

5.3. Failures

A client and a server judge a response differently, and so do their spans:

Listing 8. A missing page

```
Recording recording = Recording.builder().directory(dir, "yml").build();
try (recording) {
    Harness.inServer(
        recording.getOpenTelemetry(),
        "Missing",
        routes -> routes.createContext("/", exchange -> respond(exchange, 404, "Not found")),
        server -> server.get("/nothing"));
}
Span client = roots(recording).getFirst().getChildren().getFirst();
assertThat(client.getStatus().getCode()).isEqualTo(StatusCode.STATUS_CODE_ERROR); ①
Span server = client.getChildren().getFirst();
```

```
assertThat(server.getStatus().getCode()).isEqualTo(StatusCode.STATUS_CODE_UNSET); ②
```

① For the client, any status of 400 or more is an error.

② For the server, a 4xx is the client's error, not the server's, and leaves the status unset; a 5xx, or an exception in the handler, is the server's error.

An exception thrown by the body is recorded on the body's span, which then has the error status, and rethrown.

5.4. Other clients

The harness's client sends the trace context itself. Other clients, a browser, a command started in another process, a script, need the headers handed to them:

Listing 9. The headers that carry the trace

```
Harness.inServer(recording.getOpenTelemetry(), "Headers", routes -> {}, server -> {
    String traceId = server.span().getSpanContext().getTraceId();
    assertThat(server.propagationHeaders(Context.current()))
        .hasEntrySatisfying("traceparent", value -> assertThat(value).contains(traceId));
    return null;
});
```

With W3C propagators the headers are `traceparent`, and `baggage` when there is baggage. Without an SDK there is no valid span to propagate, and the map is empty. The Playwright module adds the headers to the browser's requests to the server, and only to those.

Chapter 6. Recording

`Recording` is in the runtime's telemetry model exporters, `org.nasdanika.sdk.runtime.models.telemetry.exporters`, not in the harness, because it is not about servers: the runtime's command line launcher records every command it runs with it.

6.1. What it is

An OpenTelemetry SDK with three model exporters:

- spans go to a traces resource, nested in their parents;
- log records go to the spans they were emitted in, and the rest to a logs resource;
- metrics go to a metrics resource.

W3C trace context and baggage propagators are set, so a recording can continue a trace passed in from another process and pass its own on. Closing the recording shuts the SDK down, which exports what is pending and saves the resources.

6.2. Building one

`Recording.builder()` takes resource attributes, `serviceName`, `serviceVersion` and any `attribute`, and where to record:

- `directory(dir)` records to `traces.xml`, `logs.xml` and `metrics.xml` in the directory, as XMI, which needs no other module.
- `directory(dir, extension)` chooses the format by extension: `yml` or `json` with the runtime's `emf-json` module, `ebin` or `egz` for EMF's binary format, plain or compressed.
- `resources(traces, logs, metrics)` records to any three EMF resources.

6.3. Using one

`getOpenTelemetry()` is the instance to pass to the harness, or to make current with `makeCurrent()` for a unit of work. After the recording is closed, `getTraces()`, `getLogs()` and `getMetrics()` return the models, so a test can assert on them without loading the files.

The files are models of the runtime's telemetry model, the OpenTelemetry protocol's data model in EMF. They can be loaded into any resource set configured by the runtime, queried, compared with the files of an earlier run, and transformed into other models, which is how documents are generated from them (Chapter 7).

Part III: Looking Ahead

Browsers, walk-throughs, and the videos and documents generated from traces.

Chapter 7. What comes next

7.1. Browsers

The runtime's Playwright module drives a browser against a harness server, in the server's trace. `Browsing.inBrowser(server, session → ⋯)` runs a browser session in a span; each step of the session is a child span; the browser's requests to the server carry the step's span, so the server's spans are children of the step that caused them; and screenshots taken before and after each step are log records in the step, with the file's path as an attribute, and so is narration: what a person would say about the step, the script of a video. It is **AVAILABLE**, and its Zephyr Book is *Walk-throughs with Playwright*.

7.2. Documents from traces

A recorded trace with screenshots is a walk-through: steps, what each step looked like, and what the server did for it. **PLANNED**:

- **Narrated videos**, rendered from the screenshots and the narration records of a walk-through in their order: each screenshot a frame held for as long as its narration takes to speak, the narration spoken by a text-to-speech service, subtitles from the same text.
- **Walk-throughs in AsciiDoc and Markdown**, a section per step with its narration, its screenshots and the requests it made, generated from the trace after the test.
- **Slide decks**, a slide per step, through the runtime's presentation and PowerPoint models.
- **Sequence diagrams** of a step, from the tree of client and server spans, through the runtime's diagram model and its layout.
- **Comparisons** of the traces of two runs: a request that appeared, a request that got slower, a step whose screenshot changed.

7.3. Beyond one JVM

A server in another process continues the trace if it reads `traceparent`, and its telemetry can be recorded by its own recording, or exported to a collector. The runtime's launcher already continues a trace from the `TRACEPARENT` environment variable, so a command started by a test can join the test's trace.

Appendix A: Reference

A.1. Types

Type	Purpose
`Harness`	`inServer(name, routes, body)` with the current OpenTelemetry, and `inServer(openTelemetry, name, routes, body)`
`Harness.ServerBody<T, E>`	The body: `T run(Server server) throws E`
`Server`	The server of a body, and a client for it; see Chapter 4
`Recording`	In `org.nasdanika.sdk.runtime.models.telemetry.exporters`: an OpenTelemetry SDK that records to the telemetry model (Chapter 6)

A.2. Spans

Span	Kind	Attributes
The body, named by the caller	`INTERNAL`	`url.full` (the base), `server.address`, `server.port`
`<method> <path>`, a request sent through the `Server`	`CLIENT`	`http.request.method`, `url.full`, `server.address`, `server.port`, `http.response.status_code`
`<method> <context path>`, a request handled by the server	`SERVER`	`http.request.method`, `http.route`, `url.path`, `url.scheme`, `server.address`, `server.port`, `client.address`, `network.protocol.version`, `user_agent.original`, `http.response.status_code`, `http.response.body.size`

Status: a client span has the error status for a response of 400 or more, or a failure; a server span for a 5xx, or a failure of the handler; the body's span for an exception thrown by the body.

Appendix B: Building this book

This book is AsciiDoc in `harness/src/docs/asciidoc`, built with the module after its tests have passed:

```
mvn -pl harness verify                                # the module, its tests and this book
mvn -pl harness verify -Dzephyr.editor-notes=false    # without editor's notes
mvn -pl harness verify -Dasciidoctor.skip=true        # without the book
```

The PDF, the single-page HTML and the EPUB are written to `target/book`; the PDF and the HTML are attached to the module's artifact with the classifier `book`, and the test sources with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/harness/tests` and of the module's `module-info.java`. The tests record their telemetry in `target/telemetry`, one directory per test, where the YAML files can be read after a build. A listing line holds at most 96 characters, and a warning fails the build.

Appendix C: Glossary

Body

The code `Harness.inServer` runs while the server is up, in a span.

Client span

A span of kind `CLIENT`: a request sent, from the sender's side.

Propagation

Carrying a trace context from one process or component to another, in request headers such as `traceparent`.

Recording

An OpenTelemetry SDK that exports to the runtime's telemetry model and saves the model when closed (Chapter 6).

Routes

The code that creates a server's contexts, the handlers for its paths.

Server span

A span of kind `SERVER`: a request handled, from the receiver's side. A child of the client span that sent the request when the context was propagated.

Span

One operation in a trace, with a name, a kind, a start and an end, attributes, a status, log records and child spans.

Trace

The tree of spans of one unit of work, identified by a trace id.

Unit of work

A test, a command or a request, with its own OpenTelemetry instance current.

Walk-through

A recorded browser session: steps, their screenshots, and the server's work for each.